

Grail Grove

Solana program security review: non-custodial 1-for-1 Chimpions NFT swap

Repository	thechimpions/grail-grove
Commit reviewed	393a4a4 ("Move to own repo for audit")
Re-audit	branch <code>audit-remediation</code> at 8378197 plus the third pass in section 9 (remediation of every code-level finding, executed verification, Squads handoff)
Program ID	GrA1LdMTRsZPrLZLjbZbQ9Cx4XNbyL1bz7enprBkDv49 (mainnet; the review began under 6hXfvd34... , re-keyed before deploy, code unchanged)
Framework	Anchor 0.32.x, mpl-token-metadata 5.1.1, spl-token 8.0
Review date	4 October 2026 (initial review, re-audit and third pass)
Reviewer	Claude (automated code review with manual reasoning)

TL;DR: how listed NFTs are protected

- **Nobody can take a listed Chimpion.** The NFT never leaves the owner's wallet. It is frozen with the per-mint Listing PDA as its only delegate, and that PDA signs only inside `delist` (owner), `swap` (atomic 1-for-1 plus fee) and `eject` (admin, back to the owner). SPL Token refuses transfer, burn, approve, revoke, owner change and close on a frozen account; Token Metadata thaws only for the signing delegate.
- **Every swap account is pinned.** Lister, treasury, listing, both token accounts, master edition and the offered NFT's metadata are all constrained; the offered NFT must be a verified, legacy Chimpion the taker actually holds; both transfers and the fee happen in one instruction or not at all. The taker's fee is capped by the `max_fee_lamports` they sign; the lister's split is fixed when they list.
- **The admin cannot drain.** Admin can pause, set fees, change the treasury and eject, and eject only ever returns the NFT to its owner. Admin bootstrap is bound to the program's upgrade authority, so it cannot be front-run, and handoff is two-step.
- **The one remaining power is the upgrade authority**, because new code could sign as any Listing PDA. It is hardened with Squads: the hot wallet deploys, the vault must execute `accept_authority` before anything irreversible happens (proof it can sign), the upgrade authority moves by Squads Safe Authority Transfer (vault and deployer co-sign; no signer check is skipped), every later upgrade is a vault-owned buffer executed from a Squads proposal that members can verify byte-for-byte, and the vault should carry a time lock so listers can exit before any upgrade runs. Immutability is available later by setting the upgrade authority to none.
- **Status.** No critical or high findings. Every code-level finding is fixed on `audit-remediation`. The third pass (section 9) executed what the earlier passes could only read: the 60-case integration suite passed on the final binary, the mainnet facts were confirmed against live data, and the deploy script was readied for the 5-of-9 Squads Safe Authority Transfer. Still open and operational: the Squad time lock and a verifiable build.

Executive summary

The on-chain program is small (eight instructions, roughly 600 lines of Rust) and well constructed. Every account is constrained by seeds, owner, type or address, the admin bootstrap is bound to the program's upgrade authority, authority transfer is two-step, fee math is overflow-safe, and the freeze-plus-delegate escrow is applied atomically. I found **no path for a third party to drain listed NFTs, steal fees, or take over the admin role** through the program itself.

The material risk sits around the program rather than in it. The program is upgradeable, so whoever holds the upgrade authority can replace the code and sweep every frozen listing, and the mainnet deploy script hands both that authority and the config admin to a pasted address with no proof that anyone can sign for it. Those two items are rated Medium. Everything else is Low or Informational: defense-in-depth checks the program currently gets for free from SPL Token semantics, a leftover delegate approval after admin eject, treasury account hygiene, and operational hardening of the scripts.

0 CRITICAL	0 HIGH	2 MEDIUM	6 LOW	7 INFORMATIONAL	60 TESTS EXECUTED
---------------	-----------	-------------	----------	--------------------	-------------------------

ID	Severity	Title	Area
M-01	Medium	Upgrade authority has unilateral control over every listed NFT	Trust model / deployment
M-02	Medium	Deploy script hands admin and upgrade authority to an unverified address, irreversibly	scripts/deploy-mainnet.ts
L-01	Low	Swap and list rely on SPL Token frozen-account semantics instead of explicit checks	swap.rs, list.rs
L-02	Low	Eject leaves a live SPL delegate approval on the owner's token account	eject.rs
L-03	Low	Treasury is not required to be a system-owned account; misconfiguration halts all swaps	initialize.rs, update_config.rs
L-04	Low	Fee parameters change instantly and apply retroactively to open listings	update_config.rs, swap.rs
L-05	Low	Helius API key is passed on child-process command lines	scripts
L-06	Low	Admin CLI defaults to mainnet and the default Solana keypair	scripts/admin.ts
I-01	Info	Listings are open offers with no expiry or trait filter (adverse selection)	Design
I-02	Info	Unverified mainnet assumptions: collection token standard and freeze authority	Deployment
I-03	Info	No verifiable-build step; declared and resolved Anchor versions differ	Build / supply chain
I-04	Info	<code>active_listings</code> uses saturating subtraction	delist.rs, swap.rs, eject.rs
I-05	Info	Misleading error code on eject owner mismatch	eject.rs

I-06	Info	Test coverage gaps	tests/
I-07	Info	Dependency and advisory notes	Cargo.lock

1. Scope and method

Everything in the repository was read in full: the Anchor program (`programs/grail-grove/src` , all eight instruction handlers, state, errors, events, fee math and metadata validation), the three operational scripts (`admin.ts` , `deploy-mainnet.ts` , `devnet-setup.ts`), the IDL sync script, the Mocha integration suite (43 cases) and its helpers, `Anchor.toml` , both `Cargo.toml` files, `Cargo.lock` , `package.json` and `.gitignore` .

The review focused on the attack classes that matter for a Solana NFT escrow:

- Missing or weak account validation: signer, owner, PDA seeds and bump, program-ID pinning, type cosplay, account substitution and duplicate-mutable accounts.
- Privilege: admin bootstrap front-running, authority takeover, re-initialization, upgrade-authority exposure.
- Asset safety: any way to move a listed NFT without a matching swap, to receive a listed NFT without surrendering one, to collect fees without delivering, or to brick a listing so the owner cannot recover it.
- Token-program semantics: what SPL Token and Token Metadata permit on a frozen, delegated token account, verified against the upstream sources rather than from memory.
- Arithmetic, rent and lamport flows; pause semantics; CPI signer-seed leakage.
- Operational security of deployment, key handling and secrets.

Verification performed in this environment: the Rust unit tests compile and pass (8 of 8). The upstream SPL Token processor and Metaplex freeze and thaw processors were fetched and inspected to confirm the invariants the program depends on. The Anchor integration suite was not executed here because it needs a local validator with a mainnet clone of Token Metadata, which this container's network policy blocks. Mainnet RPC was also blocked, so on-chain state (collection, treasury, program) could not be inspected; see I-02.

2. System overview

A holder lists a Chimpion without giving up custody: the `list` instruction approves the per-mint `Listing` PDA as SPL delegate for one token, then the PDA calls Token Metadata's `FreezeDelegatedAccount` so the mint's master edition (the freeze authority) freezes the holder's token account. A taker calls `swap` with any other verified Chimpion: they pay a flat fee that is split between the treasury and the lister, the PDA thaws the listed account and delegate-transfers the NFT to the taker, and the taker transfers their NFT to the lister. `delist` (owner) and `eject` (admin) thaw and close. Admin is a Squads vault with pause, fee and treasury controls and a two-step handoff.

Security invariant the whole design rests on

While a listing exists, the listed token account is frozen and its delegate is the Listing PDA. I confirmed against the SPL Token processor that a frozen account rejects `Transfer` , `Burn` , `Approve` , `Revoke` and `SetAuthority` , and that `CloseAccount` requires a zero balance. Only the master edition PDA can thaw, and Token Metadata only signs that thaw when the current delegate signs (`assert_signer(delegate)` plus `assert_delegated_tokens`). The Listing PDA signs only inside `delist` , `swap` and `eject` , each of which closes the listing. So between `list` and one of those three, nobody can move, re-delegate, re-own or close the listed account. That invariant holds in the code as written.

3. Attack scenarios examined

Scenario	Outcome
Front-run <code>initialize</code> after deploy to seize admin	Blocked Payer must equal <code>ProgramData.upgrade_authority_address</code> , and the <code>ProgramData</code> account is bound to the program via <code>programdata_address()</code> , so a foreign <code>ProgramData</code> cannot be substituted.
Re-initialize or close the <code>Config</code> PDA	Blocked <code>init</code> fails on an existing account; no instruction closes or reallocates config.
Set <code>authority</code> or <code>pending_authority</code> without the current authority	Blocked <code>has_one = authority</code> on propose, update, eject; accept requires the pending key to sign and be non-default.
Thaw or transfer a listed NFT from outside the program	Blocked Requires the Listing PDA signature or master edition signature; neither is obtainable.
Change the owner of the listed token account, then swap or delist with the new owner	Blocked <code>SetAuthority(AccountOwner)</code> is rejected on frozen accounts (spl-token processor line 436).
Swap with a substituted <code>lister</code> , <code>treasury</code> , listing or token account	Blocked <code>listing.owner == lister</code> , <code>address = config.treasury</code> , listing seeds bound to <code>listed_mint</code> , <code>lister_listed_token_account == listing.token_account</code> .
Receive the listed NFT without surrendering one (fake, fungible, pNFT or non-collection offer)	Blocked Offered mint must have supply 1 and 0 decimals; metadata PDA and owner enforced; mint match, legacy standard and verified collection asserted; taker signs the transfer and must hold exactly one.
Offer an NFT that is itself listed (frozen) or the same mint	Blocked Transfer fails on frozen source; <code>SameMint</code> constraint.
Lister collects the fee share but keeps the NFT	Blocked Fee transfer and delegate transfer are in one instruction; any failure reverts both.
Admin raises the fee between quote and execution	Blocked <code>max_fee_lamports</code> slippage guard, provided the client passes the displayed fee (see L-04).
Lister front-runs a swap by delisting and relisting, or moving the NFT to an accomplice who relists	Blocked The taker's transaction pins owner, token account and mint; any change fails the constraints.
Grief a listing by funding or pre-creating the Listing PDA or ATAs	Harmless Anchor <code>init</code> tolerates pre-funded PDAs; only the ATA program can create ATAs; lamports sent to the listing are refunded on close.
Signer-seed leakage via CPI to an attacker program	Blocked Token, ATA and System programs are typed; Token Metadata is pinned by address.
Arithmetic overflow in fee split or counters	Blocked u128 checked math, bps capped at 10,000, fee capped at 1 SOL, <code>overflow-checks = true</code> .
Use pause to trap listers' NFTs	Blocked <code>delist</code> and <code>eject</code> work while paused.

Upgrade the program to a malicious version and sweep listings	Possible for the upgrade authority See M-01.
Deploy with a mistyped multisig address	Unrecoverable See M-02.

4. Findings

Medium

M-01. Upgrade authority has unilateral control over every listed NFT

Anchor.toml (`upgradeable = true`), scripts/deploy-mainnet.ts steps 4 and 5, program-wide

Description

The program is deployed upgradeable and the upgrade authority is handed to the Squads vault. An upgrade can replace the program with code that signs as any Listing PDA and delegate-transfers every frozen, listed NFT to an arbitrary wallet in a single transaction per listing. Nothing in the program limits this, because by design the Listing PDA is the only key that can thaw and move a listed token. The config admin, by contrast, is well scoped: `eject` only ever returns the NFT to its owner and no admin instruction can redirect an NFT.

Impact

Compromise of the vault's signers (or a collusion at threshold) means total loss of all open listings. Users are told the swap is non-custodial, but in practice custody is "anyone who can upgrade the program".

Recommendation

- Document the vault's member count and threshold publicly, and use a Squads time lock on the vault so an upgrade proposal is visible for a period during which listers can delist.
- Publish a verifiable build (`solana-verify` / Anchor verified builds, see I-03) so the deployed bytecode can be matched to this reviewed commit.
- Once the program has been live and stable, consider making it immutable (set the upgrade authority to none). The config admin still retains pause, fee and eject, which is enough for day-to-day operation.
- Keep the open-listing count small relative to the collection by nudging listers to delist when idle (see I-01 on expiry).

Medium

M-02. Deploy script hands admin and upgrade authority to an unverified address, irreversibly

scripts/deploy-mainnet.ts lines 83, 159–167, 217–241, 265–278

Description

The `--multisig` flag is parsed as a public key and then used three times: as `Config.authority` inside `initialize`, as the IDL authority, and as the program upgrade authority with `--skip-new-upgrade-authority-signer-check`. The only sanity check is a warning if the key is on the ed25519 curve. The "verify" step compares the on-chain values back to the same command-line input, so a typo, a settings-account address pasted instead of the vault address, or a vault from the wrong Squads deployment all pass verification.

Impact

If the address is wrong, nobody can sign for it. `Config.authority` can only be changed by itself via `propose_authority`, so the program can never be paused, re-priced or unwound. The upgrade authority is lost with it, so the program also cannot be fixed. There is no recovery path short of abandoning the program ID, and any listings created in the meantime can only be closed by their owners via `delist`.

Recommendation

- Before anything irreversible, prove control of the address: fetch the Squads multisig settings account the operator names, derive the vault PDA from it and the vault index, and require equality with `--multisig`. Better still, require the vault to have executed at least one transaction on mainnet (for example a 0 SOL transfer to itself) and check for it.
- Make the config handoff recoverable: initialize with the deployer as `authority`, then run `propose_authority(vault)` and have the vault execute `accept_authority`. That exercises the vault's signing path before the upgrade authority is moved, and a bad address simply fails to accept. The script's current rationale ("deployer is never admin, not even briefly") trades a few minutes of deployer admin for an unrecoverable failure mode; with a dry run and a verified build the trade is worth making.
- Only after the vault has accepted config authority, transfer the upgrade authority, and run `solana program show` plus `pnpm admin show` as the final check.

Low

L-01. Swap and list rely on SPL Token frozen-account semantics instead of explicit checks

swap.rs lines 66–72; list.rs lines 39, 64–70

Description

`swap` checks that `lister_listed_token_account` is the account recorded at list time and holds one token, but not that its `owner` is still `lister` or that its `mint` is `listed_mint`. `list` checks the owner's token account but, unlike `swap`'s check on the offered mint, does not assert `mint.supply == 1 && mint.decimals == 0`. Both are safe today: a frozen token account cannot change owner or mint, and a Token Metadata edition account cannot exist for a mint that was not single-supply when the edition was created. The safety is inherited from two external programs rather than stated in this one.

Impact

None currently. The risk is latent: a future change to SPL Token semantics, a future instruction in this program that thaws without closing, or a Token-2022 variant would silently remove the protection.

Recommendation

Add the cheap explicit constraints: in `swap`, `lister_listed_token_account.owner == lister.key()` and `.mint == listed_mint.key()`; in `list`, `mint.supply == 1 && mint.decimals == 0`, and optionally `mint.freeze_authority == Some(master_edition.key())` to fail with a clear error rather than a CPI error.

Low

L-02. Eject leaves a live SPL delegate approval on the owner's token account

eject.rs lines 14–18, 80–98

Description

After `eject`, the owner's token account is thawed and the listing closed, but the account still carries `delegate = Listing PDA, delegated_amount = 1`. The code comments acknowledge this and argue it is inert because no instruction can sign for a PDA whose listing account no longer exists. That reasoning is correct for the current code: the three signing instructions all require a live `Listing` whose `token_account` matches, and `list` re-approves anyway.

Impact

Wallets and security tools display the approval as an active third-party spend permission on the NFT, which will generate support load and distrust. It also sets a trap for future maintainers: if any later version adds an instruction in which the Listing PDA signs a token transfer without the `listing.token_account` binding, every previously ejected account becomes drainable.

Recommendation

The program cannot revoke on the owner's behalf. Surface the state in the web app after an `eject` with a one-click SPL `Revoke`, document it in the user-facing FAQ, and add a code comment on `swap`'s `lister_listed_token_account` constraint noting that it is the control that keeps stale approvals inert.

Low L-03. Treasury is not required to be a system-owned account; misconfiguration halts all swaps

initialize.rs lines 31–37; update_config.rs lines 25–31; swap.rs lines 137–148

Description

The only on-chain constraint on the treasury is "not the default key and has at least one lamport". The deploy script checks system ownership off-chain, but `update_config` accepts any funded account. If the admin sets the treasury to an executable account, a PDA of another program, or a token account, the system-program transfer in `swap` either fails (executable accounts cannot receive lamports) or sends fees somewhere they cannot be spent. Separately, if the treasury is a normal wallet that is later drained to zero, a swap whose treasury share is below the rent-exempt minimum for a zero-data account (890,880 lamports) fails the runtime rent check. At the default 0.02 SOL fee and 50% split the share is 10,000,000 lamports, so this only bites with fees under roughly 0.0018 SOL.

Impact

Admin-only, so not an exploit, but a plausible misconfiguration that stops every swap until corrected, and fees could be permanently stranded.

Recommendation

Add `owner == system_program::ID` to both treasury constraints and require `lamports() >= Rent::minimum_balance(0)`. Keep the deploy-script check as well.

Low L-04. Fee parameters change instantly and apply retroactively to open listings

update_config.rs; swap.rs lines 133–135; state.rs lines 8–10

Description

The admin can move `swap_fee_lamports` anywhere up to 1 SOL and `treasury_bps` anywhere up to 100% in one transaction. The lister's share is computed at swap time from the current config, so a lister who posted when the split was 50/50 may receive nothing if the split is 100/0 when a taker arrives. Takers are protected by `max_fee_lamports`, but only if the client sets it to the fee it displayed rather than a generous ceiling.

Impact

Trust and UX rather than theft: the admin is a multisig and the cap is 1 SOL. A 0.02 SOL to 1 SOL jump would still surprise anyone whose client passes `u64::MAX` as the max.

Recommendation

In the web app, pass exactly the quoted fee as `max_fee_lamports`. On-chain, consider either snapshotting `treasury_bps` into the `Listing` at list time, or a Squads time lock on config changes so listers can react.

Low L-05. Helius API key is passed on child-process command lines

scripts/deploy-mainnet.ts lines 207–241; scripts/devnet-setup.ts line 333

Description

Both scripts build `anchor deploy`, `anchor idl set-authority` and `solana program set-upgrade-authority` command strings that embed the full RPC URL including `?api-key=...`, and run them with `execSync`. The key is visible to any process on the machine via `ps`, lands in CI logs that echo commands, and the shell-splitting `execSync` path means an RPC URL containing shell metacharacters would be interpreted. Console output is correctly redacted; the subprocess argv is not.

Recommendation

Pass the RPC URL to the Solana tooling through its config file or environment (`ANCHOR_PROVIDER_URL`, `solana config set --url` in a temporary config dir) instead of argv, and use `spawnSync` with an argument array rather than a shell string.

Low L-06. Admin CLI defaults to mainnet and the default Solana keypair

scripts/admin.ts lines 111–152

Description

With neither `--cluster` nor `--rpc`, the CLI targets `api.mainnet-beta.solana.com`, and without `--keypair` it signs with `~/.config/solana/id.json`. A developer who forgets a flag while iterating on devnet sends a signed mainnet transaction. Today only `initialize` could succeed with a non-multisig key, and only from the deployer, but the default still points the sharp end at production.

Recommendation

Require `--cluster` or `--rpc` explicitly, and require `--keypair` whenever a transaction will be sent. Print a confirmation prompt when the resolved cluster is mainnet and `--print` is not set.

Info I-01. Listings are open offers with no expiry or trait filter (adverse selection)

state.rs `Listing`; list.rs

A listing says "swap my Chimpion for any verified Chimpion". A rational taker always offers the least desirable one they hold, so listers should expect to receive the floor, and the fee share is their compensation. `created_at` is recorded but never used; listings never expire, so forgotten listings remain filled-at-floor targets indefinitely (and enlarge the exposure described in M-01). Consider an optional expiry checked in `swap`, and make the "any Chimpion" semantics prominent in the UI.

Info I-02. Unverified mainnet assumptions: collection token standard and freeze authority

deploy-mainnet.ts lines 40–41; metadata.rs lines 21–24

The program only works for legacy NonFungible (or pre-standard) NFTs whose mint freeze authority is the master edition PDA. Mainnet RPC was blocked from this environment, so I could not confirm that the Chimpions collection `2k8iNEAB...` matches, that the treasury `Df7VuBka...` is a system wallet, or whether the program ID is already deployed. Before mainnet, confirm a sample of Chimpions have `token_standard` `None` or `NonFungible` and `freeze_authority == master_edition`; otherwise `list` fails for every token. The Anchor test suite already covers the pNFT rejection path for the offered side only.

Info I-03. No verifiable-build step; declared and resolved Anchor versions differ

deploy-mainnet.ts; programs/grail-grove/Cargo.toml; Cargo.lock

The deploy script records the binary's SHA-256 but nothing ties it to a reproducible build of a public commit. `Cargo.toml` declares `anchor-lang = "0.32.1"` while `Cargo.lock` resolves 0.32.2. Pin exact versions (`=0.32.2`), build with `solana-verify build` or `anchor build --verifiable`, and upload the verification PDA so explorers show the program as verified. This is the main mitigation for the trust gap in M-01.

Info I-04. `active_listings` uses saturating subtraction

delist.rs line 93; swap.rs line 216; eject.rs line 101

`saturating_sub(1)` hides any future accounting bug (a double decrement would read as zero rather than fail). The counter is informational, so this is a maintainability note: `checked_sub` with an error, matching the `checked_add` in `list`, would surface drift immediately.

Info I-05. Misleading error code on eject owner mismatch

eject.rs line 34

`has_one = owner @ GrailGroveError::TokenAccountMismatch` reports a token-account error when the `owner` account does not match the listing. Introduce a dedicated `OwnerMismatch` error so the admin CLI's failure messages are accurate.

Info I-06. Test coverage gaps

tests/grail_grove.test.ts

The 43 integration cases cover the main paths and most negative cases well. Missing cases worth adding, each of which should fail today for the right reason:

- Listing a pNFT (only the offered-side pNFT rejection is tested).
- An NFT whose `collection.key` is correct but `verified` is false (the existing "outside the collection" test uses no collection at all).
- Swap with a substituted `lister_listed_token_account` or `listed_master_edition`.
- Swap where the taker's ATA for the listed mint already exists, and where the lister's ATA for the offered mint already exists.
- After `eject` : owner transfers the NFT out and the new holder lists it (exercises the stale-delegate reasoning in L-02).
- `propose_authority(Pubkey::default())` as a cancellation, followed by a rejected accept.
- An NFT whose mint freeze authority is not the master edition (list should fail cleanly).

Info I-07. Dependency and advisory notes

Cargo.lock, package.json

Resolved: anchor-lang 0.32.2, anchor-spl 0.32.2, mpl-token-metadata 5.1.1, spl-token 8.0.0, spl-associated-token-account 7.0.0, solana-program 2.3.0, curve25519-dalek 4.1.3 (includes the RUSTSEC-2024-0344 timing fix), borsh 0.10.4 and 1.8.1. `cargo audit` (advisory database as of this review, 276 crates scanned) reports **zero vulnerabilities** and three warnings, all transitive through the Solana SDK and not reachable in on-chain code: `bincode 1.3.3` unmaintained (RUSTSEC-2025-0141), `libsecp256k1 0.6.0` unmaintained (RUSTSEC-2025-0161), `rand 0.7.3` unsound with a custom logger (RUSTSEC-2026-0097). No action is needed beyond tracking Anchor and Solana SDK releases. The Metaplex client's `Metadata::from_bytes` does not check the account's `Key` discriminator, but the program binds the account to the metadata PDA and Token Metadata ownership, which is sufficient. JavaScript dependencies are devDependencies only; nothing from this repository ships to end users.

5. What the program does well

- **Admin bootstrap cannot be front-run.** `initialize` requires the payer to be the program's upgrade authority and binds the `ProgramData` account to the program through `programdata_address()`, the pattern Anchor recommends.
- **Two-step authority handoff** with a non-default check on accept, so a mistyped proposal cannot brick admin.
- **Every PDA is seed-checked** with a stored canonical bump; Metaplex PDAs use `seeds::program` plus an owner check; the Token Metadata program is pinned by address and Token, ATA and System are typed.
- **Collection gating is correct:** metadata PDA derived from the mint, mint equality, legacy token standard only, and `collection.verified && key == config.collection`.
- **Escrow is atomic:** approve and freeze happen in the same instruction; thaw, delegate transfer and counter-transfer happen in the same instruction; `close` refunds rent to the right party in all three exit paths.
- **Fee math** is done in u128 with checked operations, rounding dust goes to the lister, the fee is capped at 1 SOL and bps at 10,000, and the taker has a slippage guard.
- **Pause is one-directional:** it blocks new listings and swaps but never traps assets.
- **Deployment discipline:** dry-run by default, mainnet genesis check, program-ID and keypair agreement, treasury ownership check, post-deploy verification, redacted logs, and secrets kept out of git.
- `overflow-checks = true` in the release profile; Box ed accounts to keep the stack small; 43 integration tests and 8 passing unit tests.

6. Recommended order of work

1. Decide the upgrade-authority posture (M-01): time lock and published threshold now, verifiable build before launch (I-03), immutability as a later milestone.
2. Rework the deploy handoff so the vault proves it can sign before anything irreversible happens (M-02).
3. Add the defense-in-depth constraints (L-01, L-03) and the error-code and counter cleanups (I-04, I-05). These are a few lines each and do not change behavior for valid transactions.
4. Client work: pass the quoted fee as `max_fee_lamports` (L-04), show and offer to revoke stale approvals after eject (L-02), make the "any Chimpion" semantics explicit (I-01).
5. Script hardening (L-05, L-06) and the additional tests (I-06).
6. Confirm the mainnet assumptions in I-02 against live data before deploying.

7. Remediation status (re-audit at `e0680b2`)

All code changes live on branch `audit/security-review`, pull request #2. Each was re-read against the original finding; the "verified by" column says how far this environment could check it. The Anchor integration suite and the live-cluster scripts could not be run here (no test validator with a Token Metadata clone, no RPC egress), so those rows say so explicitly.

ID	Status	What changed	Verified by
----	--------	--------------	-------------

M-01	Mitigated ops items remain	Upgrades now go through a vault-owned buffer prepared by <code>pnpm upgrade:prepare</code> and executed from a Squads proposal; <code>--check <buffer></code> lets any member compare the on-chain buffer byte-for-byte with a local build. <code>SECURITY.md</code> documents the trust model. Still to do by the team: Squads time lock and published threshold, <code>solana-verify</code> build, later immutability.	Code review, typecheck. Not run on a cluster.
M-02	Fixed	Deploy is two runs. Run 1 deploys, initializes with the deployer as interim admin and proposes the vault; the vault must execute <code>accept_authority</code> in Squads. Run 2 verifies that on chain, then moves IDL and upgrade authority (or leaves the upgrade authority to a Squads Safe Authority Transfer with <code>--sat</code>). The vault must derive from the given Squads settings account and vault index; on-curve addresses are refused. Both runs resume and dry-run by default.	Code review, typecheck. Not run on a cluster.
L-01	Fixed	<code>swap</code> asserts the listed token account's owner is the lister and its mint is the listed mint. <code>list</code> asserts supply 1, decimals 0 and freeze authority equal to the master edition, with dedicated errors.	<code>cargo check</code> ; new tests written (not run).
L-02	Documented	Explained in <code>SECURITY.md</code> ; a new test proves the stale approval is inert when the NFT changes hands and is relisted. The one-click revoke belongs in the web app, which is outside this repository.	Code review.
L-03	Fixed	Treasury must be system-owned and rent exempt on both <code>initialize</code> and <code>update_config</code> ; tests cover an executable account and a mint.	<code>cargo check</code> ; tests written.
L-04	Fixed client item remains	<code>Listing.treasury_bps</code> is snapshotted at list time and used by <code>swap</code> ; a test changes the split to 100% after listing and checks the lister still receives their share. The web app must still pass the quoted fee as <code>max_fee_lamports</code> .	<code>cargo check</code> ; test written.
L-05	Fixed	Solana/Anchor CLIs run via <code>spawnSync</code> with the RPC URL in a private temporary config file (<code>scripts/lib/solana-cli.ts</code>). The IDL step uses a keyless cluster alias.	Code review, typecheck.
L-06	Fixed	<code>admin.ts</code> requires <code>--cluster</code> or <code>--rpc</code> , requires <code>--keypair</code> to send, and asks for interactive confirmation before any mainnet send.	Code review, typecheck.
I-01	Documented	"Any Chimpion" semantics stated in <code>SECURITY.md</code> . No on-chain expiry was added; it remains a product decision.	n/a

I-02	Pending	<code>pnpm verify:mainnet</code> checks a sample of collection members for legacy standard, master edition and freeze authority, the treasury's ownership and rent, and the program/config state. It could not be run from this environment because mainnet RPC is blocked; run it locally before deploying.	Typecheck only.
I-03	Open	No verifiable-build step added and Anchor is still declared as a caret range. Build releases with <code>solana-verify</code> and <code>pin =0.32.2</code> .	n/a
I-04	Resolved with a change of mind	<code>swap</code> uses <code>checked_sub.delist</code> and <code>eject</code> deliberately keep <code>saturating_sub</code> : they are recovery paths, and an informational counter must never be able to trap an NFT. The first remediation pass made all three checked; the re-audit reversed the two recovery paths.	<code>cargo check</code> .
I-05	Fixed	<code>eject</code> reports <code>OwnerMismatch</code> .	Test updated.
I-06	Fixed	Ten new integration cases (53 total): non-system treasury, handoff cancel, pNFT listing, unverified collection member, editionless token, substituted token account and master edition, pre-existing ATAs, fee-split snapshot, eject then transfer then relist. New mint helper for editionless tokens.	Typecheck. Not executed.
I-07	No action	<code>cargo-audit</code> clean; three transitive unmaintained-crate warnings from the Solana SDK.	<code>cargo audit</code> .

8. Re-audit notes

Scope of the second pass

Every file touched by the remediation commits was re-read with the same adversarial questions as the first pass: can any new constraint be bypassed, does any new code path let the Listing PDA sign for something it should not, can the new deploy flow leave authority somewhere unintended, and do the scripts handle partial failure. The on-chain diff is small (about 60 lines) and purely additive in checks; no instruction gained a new signer or a new CPI target.

Observations on the new code

- **Constraint ordering.** `list`'s mint constraint references `master_edition`, which is declared later in the struct. Anchor resolves this dependency and the program compiles; the integration test for an editionless token exercises exactly this constraint.
- **Fee split snapshot.** Only the split is snapshotted. The flat fee still comes from config so that the admin can lower it for everyone at once; the taker's `max_fee_lamports` remains the taker's protection. A lister who wants a different split must delist and relist.
- **Treasury check is rent-aware.** `Rent::get()?.minimum_balance(0)` is evaluated inside the constraint, so the threshold tracks the cluster's rent parameters rather than a hard-coded lamport figure.

- **Deploy run 1 interim admin.** Between run 1 and the vault's acceptance the deployer is config admin. That role can pause, re-price, change the treasury and eject; it cannot move an NFT or lamports to itself except the treasury share of fees, and no listings exist yet at that point. The window is bounded by the team's own speed.
- **Run 2 with `--sat`.** The script reports the upgrade authority as "pending SAT" rather than failing, and the deployer must be a Squad member to execute the SAT. Remove that membership afterwards; the script cannot do it.
- **Buffer tooling.** The buffer authority is set to the vault before the Squads "Add upgrade" step. The loader requires buffer authority to equal the program's upgrade authority, so the address Squads shows will be the vault; if it ever differs, stop and investigate.
- **Listing account layout changed (+2 bytes).** Any devnet deployment with existing listings must be reset; mainnet is unaffected because the program is not yet deployed.

Squads hardening, end to end

Superseded. These steps describe the deploy flow at the second pass. The launch flow changed in section 9: the deploy is one run with no multisig input, the config admin stays a team-held key, and the upgrade authority moves by SAT in Squads afterwards.

1. `pnpm deploy:prod --multisig <vault> --squads <settings> --keypair deployer.json --confirm`: deploy, initialize with the deployer as interim admin, propose the vault. The vault address must derive as vault N of the settings account under the Squads v4 program.
2. In Squads, import and execute the printed `accept_authority` transaction. The vault is now the swap admin and has proven it can sign.
3. In Squads, Programs → Add Program → Create SAT; approve to threshold; execute with the deployer wallet (temporarily a member).
4. `pnpm deploy:prod --finalize --sat ... --confirm`: transfers the IDL authority and verifies admin, upgrade and IDL authority all sit on the vault.
5. Every release: `pnpm upgrade:prepare`, then Programs → Add upgrade in Squads with the buffer, spill address and commit link; members run `pnpm upgrade:prepare --check <buffer>` and compare with a `solana-verify` build before approving; the time lock gives listers a window to delist.

What this environment could not verify

- The Anchor integration suite (53 cases) was typechecked but not executed. Run `pnpm test` locally before merging.
- The deploy, upgrade, admin and verify scripts were typechecked but never run against a cluster. Rehearse the full sequence above on devnet with a devnet Squad.
- I-02 remains unconfirmed; `pnpm verify:mainnet` is the one-command check.

9. Third pass: executed verification and launch readiness

Branch `audit-remediation`, 4 October 2026, run from a workstation with a local validator (Token Metadata cloned from mainnet) and mainnet RPC. This pass did what sections 7 and 8 could not: it ran the tests and the scripts, and checked the chain.

Integration suite executed

The 60-case Mocha suite (the 53 cases from the remediation plus 7 escape attempts) was executed against a local validator with Token Metadata cloned from mainnet, on the rebuilt binary at the final program ID: 60 passing, 0 failing, 0 skipped, 50 minutes. After the crate version was set to 1.0.0 for release (which changes the binary, though not its logic), the suite was run again on that binary: all 60 cases pass. Three cases timed out in the first full run while the build machine was heavily loaded by unrelated processes; those three and the cases the run had not reached were then run on a fresh validator and passed, so every case has passed on the release binary. The Rust unit tests pass (8 of 8). This closes the "typechecked but not executed" caveat in sections 7 and 8 and verifies I-01, I-06 and the L-01, L-03 and L-04 fixes by execution. The suite now includes a block of seven escape attempts against a listed NFT, each run as its owner through Token Metadata and SPL Token rather than through the program: `UnlockV1`, `ThawDelegatedAccount` signed by the owner and by the collection update authority, `SPL Revoke` and `RevokeStandardV1`, `TransferV1`, `BurnV1` and legacy `BurnNft`. Every attempt fails with the expected upstream error and the token account stays frozen, delegated to the Listing PDA, with the listing intact. This is the invariant from section 2, observed rather than argued.

Mainnet assumptions confirmed (closes I-02)

Check	Result
Collection <code>2k8iNEAB...</code>	Exists, metadata and master edition present, token standard NonFungible, name "The Chimpions". Its own token has supply 0 (burned or never minted); the program never reads the collection mint, only each member's verified collection pointer, so this changes nothing.
Members (Helius DAS: 222, interface <code>V1_NFT</code> ; 25 sampled across the list)	25/25 legacy NonFungible, 25/25 verified in the collection, 25/25 with the master edition as the mint's freeze authority. This is exactly the shape <code>list</code> requires.
Treasury <code>Df7VuBka...</code>	System-owned wallet, rent exempt (about 45 SOL).
Program <code>GrA1LdMT...</code>	Not deployed; config PDA unused. Deploy preflight passes up to the funding check.

Upgrade authority handoff to a 5-of-9 Squads vault

The team's decision is a Squads multisig with threshold 5 of 9 holding the upgrade authority, moved by Safe Authority Transfer (SAT) run directly in Squads. Per the Squads documentation, the current authority wallet is added to the Squad as a member and executes the SAT once the threshold is met; the vault PDA and that wallet co-sign, so `--skip-new-upgrade-authority-signer-check` is never used. `scripts/deploy-mainnet.ts` was changed accordingly:

Config admin stays a team-held key, and the deploy script stays clear of the multisig. The team chose to keep `Config.authority` on the deployer key rather than the multisig, so an emergency pause or eject does not wait for 5 of 9 approvals. This revises the M-02 remediation: the vault never runs `accept_authority`. The trade-off is acceptable because the admin role cannot move an NFT except back to its owner. A compromised admin key could pause the board, redirect treasury fees, raise the fee toward the 1 SOL cap (takers still sign a maximum) and take the admin role; the multisig, as upgrade authority, can recover from all of these with an upgrade. The deploy is now a single resumable run that deploys, creates the Anchor IDL account, initializes, publishes the IDL and `security.txt`, and then verifies: the on-chain program bytes equal the local build, the admin is the deployer with no pending handoff (a leftover proposal would let that key take the role), the Anchor IDL and both Program Metadata accounts match the local files,

and the config holds the expected collection. The same checks run on their own with `--verify`. The upgrade authority is moved afterwards by SAT in Squads, which has the vault co-sign and so proves control; the script only reports who holds it. The Anchor IDL authority stays with the deployer.

- The script never moves the upgrade authority and takes no multisig input. The CLI transfer with the signer check skipped was removed.
- Confirming the Squad's threshold and time lock, and that the post-SAT upgrade authority is the Chimpions vault, are manual steps in Squads; `--verify` prints the upgrade authority so it can be compared.
- After the SAT, remove the deployer from the Squad's members. With Vote it would count toward the threshold and with Execute it could run approved proposals. The script does not check this; confirm it in Squads.

Program ID

The program was re-keyed to the vanity address `GrA1LdMTRsZPrLZLjbZbQ9Cx4XNbyL1bz7enprBkDv49` before deploy. `declare_id!`, `Anchor.toml` and the IDL were synced and the binary rebuilt; the integration suite above ran against this ID. The program keypair is stored outside version control. The earlier devnet deployment at `6hXfvd34...` is orphaned and uses the pre-remediation Listing layout; the next devnet setup deploys fresh.

Still open

- **Squad time lock** (M-01): set in Squads, published with the threshold. The deploy script reports the live value.
- **Verifiable build** (I-03): `solana-verify` is not installed on the build machine and the Anchor CLI in use is 0.31.1 while the program pins 0.32.1. Build the release binary with `solana-verify` from the tagged commit before the Squad approves anything.
- **Web client** (L-02, L-04, I-01): pass the quoted fee as `max_fee_lamports`, offer the one-click revoke after an eject, and state plainly that a listing accepts any Chimpion.
- **IDL and security.txt** (resolved in the deploy script): Anchor 0.32's IDL account creation needs only a signer, and the creator becomes its authority, so whoever creates it first owns the fixed IDL address and could publish a misleading interface. Run 1 now creates it immediately after the deploy, waiting only until the program is invocable, and refuses to continue if it already exists under another authority. Run 1 also writes the IDL and `security.json` as canonical Program Metadata accounts (`ProgM6JCC...`). Only the upgrade authority can create those, and they pass to the vault with it. Uploads were rehearsed on devnet, including the multi-transaction IDL write, and read back identical. The security contacts in `security.json` must be filled in before run 1 will start.

Appendix A. Files reviewed

Path	Notes
<code>programs/grail-grove/src/lib.rs</code>	Entry points, program ID
<code>programs/grail-grove/src/state.rs</code>	<code>Config</code> , <code>Listing</code> , constants
<code>programs/grail-grove/src/errors.rs</code> , <code>events.rs</code>	Error codes and events
<code>programs/grail-grove/src/fee.rs</code>	Fee split and 7 unit tests

programs/grail-grove/src/metadata.rs	Collection membership check
programs/grail-grove/src/instructions/*.rs	initialize, update_config, propose_authority, accept_authority, list, delist, swap, eject
scripts/admin.ts, deploy-mainnet.ts, devnet-setup.ts, sync-idl.mjs	Operations
scripts/prepare-upgrade.ts, verify-mainnet.ts, lib/solana-cli.ts, SECURITY.md	Added by the remediation; re-audited
tests/grail_grove.test.ts, tests/helpers/*.ts	43 integration cases (53 after remediation)
Anchor.toml, Cargo.toml (x2), Cargo.lock, package.json, tsconfig.json, .gitignore	Configuration

Appendix B. Upstream behaviour verified

Claim relied on	Source checked
Frozen token account rejects Approve and Revoke	solana-program/token processor.rs , process_approve / process_revoke frozen checks
Frozen token account rejects SetAuthority (owner cannot be changed)	same file, process_set_authority line 436
CloseAccount requires zero balance for non-native accounts	same file, process_close_account lines 682–683
FreezeDelegatedAccount / ThawDelegatedAccount require the delegate to sign, the delegate to match, delegated amount to equal balance, and the edition to be the mint's freeze authority	mpl-token-metadata processor/freeze/freeze_delegated_account.rs , thaw_delegated_account.rs , assertions/misc.rs::assert_delegated_tokens
Metadata::from_bytes is plain Borsh deserialization without a key check	mpl-token-metadata clients/rust/src/generated/accounts/metadata.rs

Appendix C. Severity definitions

Severity	Meaning
Critical	Direct loss of user assets or admin takeover by an unprivileged party, exploitable now.
High	Loss or takeover requiring unusual but realistic conditions, or permanent freezing of user assets.
Medium	Loss or outage requiring a privileged party's compromise or error, or a trust assumption users would reasonably object to.
Low	Defense-in-depth gaps, admin-only misconfiguration, operational hygiene.
Info	Design notes, code quality, test coverage, unverified assumptions.

This review covers the code at the stated commits only. It does not cover the web client, the Squads configuration, or the live mainnet state, none of which are in the repository.